

COSC3380-Database System-TA-Lab

Lei and Aayush

August 2024

1 Linux Distribution

Linux distribution suitable for postgres:

- Ubuntu
- Debian
- CentOS
- Fedora
- Arch linux

2 Linux Bash

Command	Description	Examples
ls	List files and directories	ls: display all files
cd	Change Directory	cd /path/to/directory
mkdir	Create a new directory	mkdir new_folder, creates a new folder with name new_folder
rm	remove files and directories	rm test.txt deletes the test.txt text file rm -r my_directory deletes the entire folder my_directory
cp	copy files and directories	cp test.txt /path/to/directory copies test.txt to directory folder cp -r my_directory path/to/directory copies entire directory to directory folder
mv	move/renames the file or directory	mv file.txt new_file.txt renaming file to new_file mv file.txt path/to/directory
cat	view the content of a file	cat text.txt display the content of file

3 Connecting to a Server Using SSH

SSH (Secure Shell) is a protocol that allows secure remote access to servers. Follow these steps to connect to a server:

1. Open your terminal or command prompt.
2. Use the following command format to initiate an SSH connection:

```
1 ssh username@hostname
2
```

Listing 1: SSH Connection Command

Replace **username** with your actual username and **hostname** with the server's address.

3. For example, to connect to the UH Computer Science server:

```
1 ssh username@bds01.cs.uh.edu
2
```

Listing 2: SSH Connection to UH CS Server

4. When prompted, enter your password. Note: For security reasons, the password characters won't be displayed as you type.

Important: If this is your first time connecting to the server, you may receive a fingerprint authentication prompt. Verify the fingerprint with your system administrator before accepting.

If you are new to SSH or need a comprehensive guide, check out this excellent tutorial:

[DigitalOcean: How To Use SSH to Connect to a Remote Server](#)

4 Creating Folders on a Server Using Command Line

To create a folder (directory) on a server using the command line, follow the following steps:

1. Once connected, you can use the `cd` command to navigate to a specific folder:

```
1 cd /path/to/your/folder
2
```

Replace `/path/to/your/folder` with the actual path to which you want to navigate.

2. To move up one directory level, use:

```
1 cd ..  
2
```

3. To return to your home directory, use:

```
1 cd ~  
2
```

or simply:

```
1 cd  
2
```

4. Use the `mkdir` command to create a new folder:

```
1 mkdir newfoldername  
2
```

Replace `newfoldername` with the desired name for your new folder.

5. To create a folder in a specific location, provide the path:

```
1 mkdir /path/to/your/new/foldername  
2
```

6. To create nested folders (multiple levels at once), use the `-p` option:

```
1 mkdir -p /path/to/your/new/nested/folders  
2
```

7. To verify that the folder was created, use the `ls` command:

```
1 ls -l  
2
```

Or, for a specific path:

```
1 ls -l /path/to/your/new/folder  
2
```

Tips:

- Use quotes for folder names with spaces: `mkdir "My Folder"`
- The `-v` (verbose) option with `mkdir` will display a message for each created directory
- Ensure you have the necessary permissions to create folders in the desired location

Remember: Unix-like systems are case-sensitive, so "Folder" and "folder" would be two different directories.

5 Upload Files via SSH

To securely transfer files from your local machine to a remote server, use the `scp` (secure copy) command. The basic syntax is:

```
1 scp [options] source destination
```

For example, to upload a file to a specific directory on the remote server:

```
1 scp cool_stuff.txt username@example.com:/path/to/destination/
```

To upload an entire directory, use the `-r` (recursive) option:

```
1 scp -r local_directory username@example.com:/path/to/
  destination/
```

For more detailed information on `scp`, visit: [SuperUser: How to upload local file to the server through the Linux terminal](#)

6 Download Files via SSH

To download files from a remote server to your local machine, reverse the source and destination in the `scp` command:

```
1 scp username@example.com:/path/to/remote/file.txt /path/to/
  local/destination/
```

To download an entire directory:

```
1 scp -r username@example.com:/path/to/remote/directory/ /path/to/
  /local/destination/
```

For a visual guide on copying files from a server, see: [YouTube: Copy files from a server using SCP](#)

7 Create Table

To create a table, we can use 'CREATE TABLE' command. Here is the syntax:

```
1 CREATE TABLE table_name (  
2     column1 datatype constraints,  
3     column2 datatype constraints,  
4     ...  
5     columnN datatype constraints  
6 );
```

Example command to create a simple table:

```
1 CREATE TABLE employees (  
2     id SERIAL PRIMARY KEY,  
3     first_name VARCHAR(100) NOT NULL,  
4     last_name VARCHAR(100) NOT NULL,  
5     email VARCHAR(100) UNIQUE,  
6     hire_date DATE NOT NULL  
7 );
```

```
1 -- Create the salary_info table  
2 CREATE TABLE salary_info (  
3     id SERIAL PRIMARY KEY,  
4     employee_id INT NOT NULL,  
5     salary DECIMAL(10, 2) NOT NULL,  
6     location VARCHAR(100) NOT NULL,  
7     effective_date DATE NOT NULL,  
8     FOREIGN KEY (employee_id) REFERENCES employees(id)  
9 );
```

```
1 -- Create the salary_info table  
2 CREATE TABLE salary_info (  
3     id SERIAL PRIMARY KEY,  
4     employee_id INT NOT NULL,  
5     salary DECIMAL(10, 2) NOT NULL,  
6     location VARCHAR(100) NOT NULL,  
7     effective_date DATE NOT NULL,  
8     FOREIGN KEY (employee_id) REFERENCES employees(id)  
9 );  
10  
11 -- Create the departments table  
12 CREATE TABLE departments (  
13     id SERIAL PRIMARY KEY,  
14     department_name VARCHAR(100) NOT NULL,  
15     employee_id INT NOT NULL,  
16     FOREIGN KEY (employee_id) REFERENCES employees(id)  
17 );
```

8 Add Data into table

To insert data into a table, you use the INSERT INTO statement. Here's the basic syntax:

```
1  INSERT INTO table_name (column1, column2, column3, ...)
2  VALUES (value1, value2, value3, ...);
```

To insert a new employee into a table:

```
1  INSERT INTO employees (first_name, last_name, email,
2  hire_date)
3  VALUES ('John', 'Doe', 'john.doe@example.com', '2024-08-27'
4  );
```

Inserting multiple rows:

```
1  -- Insert sample data into employees table
2  INSERT INTO employees (first_name, last_name, email, hire_date)
3  VALUES
4  ('John', 'Doe', 'john.doe@example.com', '2023-01-15'),
5  ('Jane', 'Smith', 'jane.smith@example.com', '2023-02-01'),
6  ('Mike', 'Johnson', 'mike.johnson@example.com', '2023-03-10'
7  );
```

```
1  -- Insert sample data into salary_info table
2  INSERT INTO salary_info (employee_id, salary, location,
3  effective_date)
4  VALUES
5  (1, 50000.00, 'New York', '2023-01-15'),
6  (2, 55000.00, 'Los Angeles', '2023-02-01'),
7  (3, 52000.00, 'Chicago', '2023-03-10');
```

Inserting data without specifying Column names:

```
1  INSERT INTO employees
2  VALUES (1, 'Mark', 'Brown', 'mark.brown@example.com', '
3  2024-08-22');
```

```
1  -- Insert example rows into the employees table
2  INSERT INTO employees (first_name, last_name, email, hire_date)
3  VALUES
4  ('John', 'Doe', 'john.doe@example.com', '2024-08-01'),
5  ('Jane', 'Smith', 'jane.smith@example.com', '2024-08-15'),
6  ('Alice', 'Johnson', 'alice.johnson@example.com', '2024-08-20')
7  ,
8  ('Mark', 'Brown', 'mark.brown@example.com', '2024-08-25'),
9  ('Emily', 'Davis', 'emily.davis@example.com', '2024-08-27');
10 -- Insert example rows into the salary_info table
```

```

11 INSERT INTO salary_info (employee_id, salary, location,
12     effective_date)
13 VALUES
14 (1, 60000.00, 'New York', '2024-08-01'),
15 (2, 70000.00, 'Los Angeles', '2024-08-15'),
16 (3, 75000.00, 'Chicago', '2024-08-20'),
17 (4, 80000.00, 'Houston', '2024-08-25'),
18 (5, 72000.00, 'Phoenix', '2024-08-27');
19
20 -- Insert example rows into the departments table
21 INSERT INTO departments (department_name, employee_id)
22 VALUES
23 ('Engineering', 1),
24 ('Human Resources', 2),
25 ('Marketing', 3),
26 ('Sales', 4),
27 ('Finance', 5);

```

9 Select Data

To retrieve the data from the table:

```

1 SELECT column1, column2, ...
2 FROM table_name
3 WHERE condition;

```

If you want to select all columns from the table:

```

1 SELECT * FROM employees;

```

To select a specific column:

```

1 SELECT first_name, last_name, email FROM employees;

```

Filtering the result by using the 'WHERE' clause:

```

1 SELECT first_name, last_name, hire_date
2 FROM employees
3 WHERE hire_date >= '2024-08-21';

```

Sorting the results by using the clause 'ORDER BY':

```

1 SELECT first_name, last_name, email
2 FROM employees
3 ORDER BY last_name ASC;

```

Combining conditions using 'AND'/'OR':

```

1 SELECT first_name, last_name, hire_date

```

```

2 FROM employees
3 WHERE first_name = 'John' AND hire_date BETWEEN '2024-08-01
  ' AND '2024-08-31';

```

Aggregating Data with functions:

```

1 SELECT COUNT(*) AS total_employees FROM employees;

```

10 Join Table

To join two tables in PostgreSQL We can use the following example to join two tables.

```

1 SELECT *
2 FROM table1
3 JOIN table2 ON table1.column_name = table2.column_name;

```

Example command to join two tables:

```

1 -- Example query to join the tables
2 SELECT e.first_name, e.last_name, s.salary, s.location
3 FROM employees e
4 JOIN salary_info s ON e.id = s.employee_id;

```

The *e* and *s* are indeed table aliases or table name aliases.

```

1 SELECT e.first_name, e.last_name, s.salary, d.department_name
2 FROM employees e
3 JOIN salary_info s ON e.id = s.employee_id
4 JOIN departments d ON e.id = d.employee_id;

```

10.1 JOIN Types

There are several types of JOINS you can use:

INNER JOIN: Returns only the rows where there's a match in both tables

LEFT JOIN: Returns all rows from the left table and matching rows from the right table

RIGHT JOIN: Returns all rows from the right table and matching rows from the left table

FULL OUTER JOIN: Returns all rows when there's a match in either table

10.2 Reference Link about SQL Joins

[w3schools link - SQL Joins](#)

11 Group by

```
1 SELECT column_name(s)
2 FROM table_name
3 WHERE condition
4 GROUP BY column_name(s)
5 ORDER BY column_name(s);
```

```
1 -- Example 1: Count employees in each location, only for
   salaries above 50000
2 SELECT location, COUNT(*) as high_salary_count
3 FROM salary_info
4 WHERE salary > 50000
5 GROUP BY location;
6
7 -- Example 2: Calculate the average salary for each location
8 SELECT location, AVG(salary) as average_salary
9 FROM salary_info
10 GROUP BY location;
11
12 -- Example 3: Find the highest salary in each location
13 SELECT location, MAX(salary) as highest_salary
14 FROM salary_info
15 GROUP BY location;
16
17 -- Example 4: Calculate total salary budget for each location
18 SELECT location, SUM(salary) as total_salary
19 FROM salary_info
20 GROUP BY location;
```

11.1 Reference Link about SQL GROUP BY

w3schools link - SQL Group by

```
1 SELECT d.department_name, AVG(s.salary) AS avg_salary
2 FROM departments d
3 JOIN salary_info s ON d.employee_id = s.employee_id
4 GROUP BY d.department_name;
```

12 two demo account for practices

No.	Username	Password
1	user_demo1	7Hy2NmK4fL9x
2	user_demo2	3Jf8RqS5vB6y

13 System available time

Days	Morning	Evening
Monday	9am–11am	6pm–11pm
Tuesday	9am–3pm	—
Wednesday	9am–11am	6pm–11pm
Thursday	9am–3pm	—
Friday	9am–11am	6pm–11pm
Saturday	9am–1pm	—
Sunday	—	—

Note: Additional 2-hour slots may be added later when homework assignments are posted.

14 psycopg2 module

Using psycopg2 module in Python to interact with PostgreSQL databases. This popular adapter allows Python applications to execute SQL commands, manage transactions, and retrieve data directly from a PostgreSQL database.

The objective of this lab is to provide hands-on experience in connecting a PostgreSQL database, executing SQL queries, and managing data in Python.

Operations we will be going through:

- Inserting
- Updating
- Deleting
- fetch and manipulating data
- handling exceptions

14.1 Prereuisites

14.1.1 Softwares

- Python
- PostgreSQL
- psycopg2 module

14.1.2 Python skills

- Basic programming concepts
- SQL

14.2 Installing psycopg2

Use the below bash command to install the module

```
1 pip install psycopg2
```

14.3 Establishing connection

The first step is to establish a connection between the Python script and the database. Here we will go through the process of setting up this connection using the psycopg2 module.

There are several key parameters needed to establish the connection. These parameters are:

- **dbname:** The name of the database you want to connect to.
- **user:** The username under which you want to connect. This user must have the appropriate permissions to access the database.
- **password:** The password associated with the username.
- **host:** The address of the machine where the PostgreSQL database is hosted. If the database is on your local machine, you can use 'localhost'.
- **port:** The port number on which the PostgreSQL server is listening. The default port is 5432.

These parameters are essential to form a connection string that psycopg2 uses to connect to the database.

A sample code to connect a PostgreSQL database:

```
1 import psycopg2
2
3 try:
4     connection = psycopg2.connect(
5         dbname="your_database",
6         user="your_username",
7         password="your_password",
8         host="localhost",
9         port="5432"
10    )
11    print("Connection successful")
12 except Exception as e:
13    print(f"An error occurred: {e}")
```

*Assume that you have a database called **your_database** running on your local machine.*

If the connection is successful, a confirmation message ("Connection successful") is printed to the console.

14.4 Executing SQL Queries

Once a connection to the PostgreSQL database has been established, the next step is to interact with the database by executing the SQL queries.

Here, we will go through creating a cursor, executing various SQL commands, and handling the results.

14.4.1 Creating a Cursor Object

A cursor allows one to execute SQL queries and fetch results. In `psycopg2`, you will create a cursor using the connection object.

```
1 cursor = connection.cursor()
```

The `cursor()` method creates a new cursor object that is used to interact with the database. It is like a pointer that we use to execute commands and retrieve data from the database. **Imp:** You can have multiple cursors in the same program, each handling different operations simultaneously.

14.4.2 Executing SQL queries

With the cursor created, you can now execute various SQL commands to interact with the database.

SELECT Query

A `SELECT` query is used to retrieve data from one or more tables in the database.

```
1 cursor.execute("SELECT * FROM your_table")
2 records = cursor.fetchall()
3 for record in records:
4     print(record)
```

The `cursor.execute()` executes the SQL query which is passed as a string. Here, we are selecting all records from the table named `your_table`.

The `cursor.fetchall()` is used to fetch all the rows returned by the query. The result is stored in the variable `records`, which is a list of tuples, each representing a row in the table.

The loop iterates through each record and prints it.

INSERT Query

An `INSERT` query adds a new record to a table in the database.

```
1 cursor.execute("INSERT INTO your_table (column1, column2)
2     VALUES (%s, %s)", (value1, value2))
3 connection.commit()
```

The **Parameterized Query** uses placeholders (%s) for the values to be inserted.

The **Passing Parameters** are the actual values (value1, value2) which are passed as a tuple to the `execute()` method.

The `connection.commit()` commits the transaction to make the changes permanent in the database.

UPDATE Query

An UPDATE query modifies existing records in a table.

```
1 cursor.execute("UPDATE your_table SET column1 = %s WHERE
   condition_column = %s", (new_value, condition_value))
2 connection.commit()
```

The **Updating Records** query updates the value in `column1` to `new_value` for all rows where `condition_column` matches `condition_value`.

DELETE Query

A DELETE Query removes a record from the table based on a condition.

```
1 cursor.execute("DELETE FROM your_table WHERE condition_column =
   %s", (condition_value,))
2 connection.commit()
```

This query deletes all rows where `condition_column` equals `condition_value`.

GROUP BY and Aggregate functions

The GROUP BY clause in SQL is used to arrange identical data into groups. This clause is often used with aggregate functions like `COUNT()`, `SUM()`, `AVG()`, `MAX()`, and `MIN()`. These functions perform calculations on set of values and return a single value per group. Example to group students by their grade and count the number of students in each grade:

```
1 cursor.execute("""
2     SELECT grade, COUNT(*)
3     FROM students
4     GROUP BY grade
5 """)
6 results = cursor.fetchall()
7 for result in results:
8     print(f"Grade: {result[0]}, Number of Students: {result[1]}")
9     "
```

JOIN Operations

JOIN operations are used to combine rows from two or more tables based on a related column between them. The most common types of JOINS are:

- INNER JOIN: Returns only the rows where there is a match in both tables.
- LEFT JOIN (or LEFT OUTER JOIN): Returns all rows from the left tables and the matched rows from the right table. If no match found, NULL is returned.
- RIGHT JOIN (or RIGHT OUTER JOIN): Returns all rows from the right table and the matched rows from the left table.
- FULL JOIN (or FULL OUTER JOIN): Returns rows when there is a match in either the left or right table.

Example of INNER JOIN to combine data from two tables, courses and students, retrieving the names of students along with the courses they are enrolled in:

```
1 cursor.execute("""
2     SELECT students.name, courses.course_name
3     FROM students
4     INNER JOIN courses ON students.course_id = courses.id
5 """)
6 results = cursor.fetchall()
7 for result in results:
8     print(f"Student: {result[0]}, Course: {result[1]}")
```

Example of LEFT JOIN to retrieve all students, including those who may not be enrolled in any course:

```
1 cursor.execute("""
2     SELECT students.name, courses.course_name
3     FROM students
4     LEFT JOIN courses ON students.course_id = courses.id
5 """)
6 results = cursor.fetchall()
7 for result in results:
8     print(f"Student: {result[0]}, Course: {result[1]}")
```

Example of FULL JOIN to retrieve all records when there is match in either the left or right table:

```
1 cursor.execute("""
2     SELECT students.name, courses.course_name
3     FROM students
4     FULL JOIN courses ON students.course_id = courses.id
5 """)
6 results = cursor.fetchall()
7 for result in results:
8     print(f"Student: {result[0]}, Course: {result[1]}")
```

14.4.3 Fetching Results

When executing SELECT queries, psycopg2 provides several methods to fetch data:

fetchall()

This retrieves all rows from the result set. Example:

```
1 records = cursor.fetchall()
```

fetchone()

This retrieves the next row of the result set, returning a single tuple. Example:

```
1 record = cursor.fetchone()
```

fetchmany(size)

This retrieves the next **size** number of rows from the result set. Example:

```
1 record = cursor.fetchmany(10)
```

14.5 Rolling back a transaction

If an error occurs during a transaction, you can roll back the changes to ensure the database remains in a consistent state.

```
1 connection.rollback()
```

This method undoes all changes made during the transaction, reverting the databases to its previous state.

14.6 Working with Data Types

Python data types maps to PostgreSQL data types and handles various types of data, including dates and times. Some common mappings are:

- Python str → PostgreSQL TEXT, VARCHAR
- Python int → PostgreSQL INTEGER
- Python float → PostgreSQL FLOAT, REAL
- Python bool → PostgreSQL BOOLEAN

- Python `datetime.date` → PostgreSQL `DATE`
- Python `datetime.datetime` → PostgreSQL `TIMESTAMP`

14.6.1 Handling Dates and Times

Inserting dates example:

```
1 import datetime
2 cursor.execute("INSERT INTO your_table (date_column) VALUES (%s
3 )", (datetime.date.today(),))
4 connection.commit()
```

This retrieves the current date and inserted into the `date_column` of `your_table`.

Handling Time zones:

```
1 from datetime import datetime, timezone
2 now = datetime.now(timezone.utc)
3 cursor.execute("INSERT INTO your_table (timestamp_column)
4 VALUES (%s)", (now,))
5 connection.commit()
```

14.7 Using Context Manager

Context Managers are allowed to manage resources such as file handles and database connections, ensuring that they are properly cleaned up after use. This allows you to allocate and release resources precisely when you want to. The most common way to use a context manager is through the `with` statement.

In `psycopg2`, context managers can automatically handle the opening and closing of database connections and cursors. Example:

```
1 with psycopg2.connect(dbname="your_database", user="
2 your_username", password="your_password") as connection:
3     with connection.cursor() as cursor:
4         cursor.execute("SELECT * FROM your_table")
5         records = cursor.fetchall()
6         for record in records:
7             print(record)
```

Connection Context Manager: The `with` statement is used to create a context manager for the database connection. When the block of code within the `with` statement is exited, whether normally or via an exception, the connection is automatically closed.

Cursor Context Manager: Similarly, the cursor is managed by another `with` statement inside the connection context. The cursor is automatically closed when the block is exited.

14.8 Closing the Connection

To close the connection, you can use the following code:

```
1 # Closing the cursor and connection
2 cursor.close()
3 connection.close()
4 print("Connection closed")
```